Week-6 | Summary

Karthik Thiagarajan

1. Common 2
    1.1. Notation 2
    1.2. Labeled Dataset 2
    1.3. Data-matrix 3
    1.4. Label Vector 3
    1.5. Data-point 3
2. MSE 3
3. Overfitting and Underfitting 4
    3.1. Toy dataset 4
    3.2. Overfitting 7
    3.3. Data Augmentation 9
    3.4. Underfitting 10
4. Model Complexity 11
5. Regularization 11
    5.1. Ridge Regression 12
    5.2. LASSO Regression 15
6. Cross Validation 16
    6.1. $k-$ Fold cross validation 16
    6.2. Leave one out cross validation 17
7. Probabilistic view 18

1. Common

1.1. Notation

Scalars:

$$x_1, x_2, y_1, y_2, z_2, z_2, a, b, \alpha, \beta$$

Column vector:

$$\mathbf{x} \in \mathbb{R}^d$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$$

Row vector:

$$\mathbf{x} \in \mathbb{R}^d$$

$$\mathbf{x}^T = \begin{bmatrix} x_1 & \cdots & x_d \end{bmatrix}$$

Matrix:

$$\mathbf{X} \in \mathbb{R}^{d \times n}$$

1.2. Labeled Dataset

$$D = \{(\mathbf{x}_1, y_1), \ \cdots, (\mathbf{x}_n, y_n)\}$$

1.3. Data-matrix

$$\mathbf{X} \in \mathbb{R}^{d \times n}$$

- $d \rightarrow$ number of features
- $n \rightarrow$ number of data-points

$$X = \begin{bmatrix} | & & | \\ \mathbf{x}_1 & \cdots & \mathbf{x}_n \\ | & & | \end{bmatrix}$$

1.4. Label Vector

$$\mathbf{y} \in \mathbb{R}^n$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

1.5. Data-point

$$\mathbf{x}_i \in \mathbb{R}^d$$

2. MSE

The label vector $\mathbf{y}$ is a random vector with mean $\mathbf{X}^T\mathbf{w}$ and covariance matrix $\sigma^2\mathbf{I}$:

$$\mathbf{y} = \mathcal{N}\left(\mathbf{X}^T\mathbf{w}, \sigma^2\mathbf{I}\right)$$

The ML estimator in linear regression is also a random vector:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}^T\right)^{\dagger}\mathbf{y}$$

For this discussion, let us assume that rank$(\mathbf{X}) = d$, so that:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}\mathbf{X}^T\right)^{-1}\mathbf{X}\mathbf{y}$$

The expected value of the estimator is $E[\widehat{\mathbf{w}}] = \mathbf{w}$, which means that this is an unbiased estimator. The MSE of this estimator is given by:

$$E\Big[||\widehat{\mathbf{w}} - \mathbf{w}||^2\Big] = \sigma^2 \cdot \text{trace}\Big[\left(\mathbf{X}\mathbf{X}^T\right)^{-1}\Big]$$

3. Overfitting and Underfitting

3.1. Toy dataset

Consider a regression problem in which the true relationship is given by:

$$g(x) = \sin 2\pi x,\ \ 0 \leqslant x \leqslant 1$$

The label is a noisy, corrupted version of this function. The noise is an error term ϵ that is sampled from a zero mean Gaussian with variance of 0.1:

$$\epsilon \sim \mathcal{N}(0, 0.1)$$

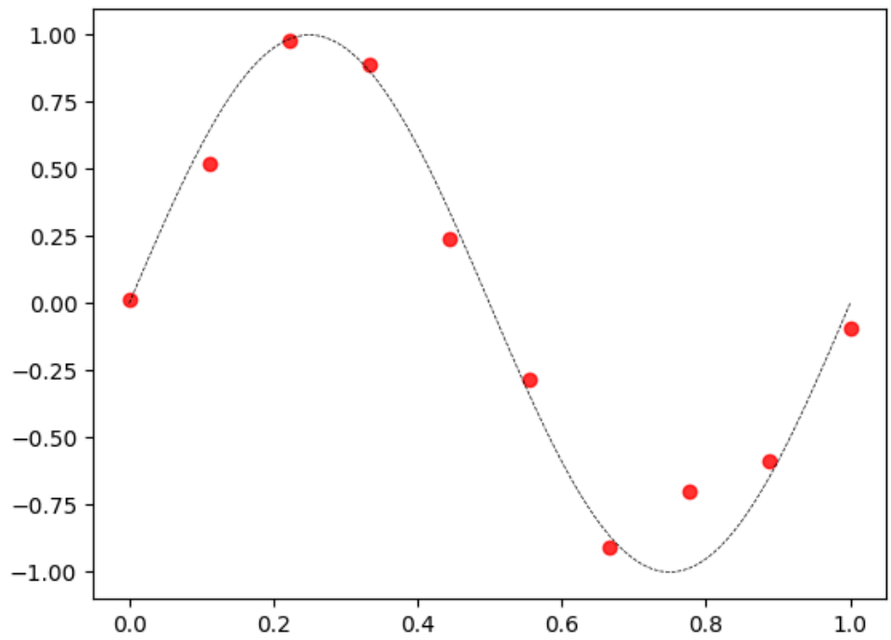
The label is therefore:

$$y_i \mid x_i \sim \mathcal{N}(g(x_i), 0.1)$$

This is the data-generation part. In reality, we don't know the functional form of $g(x)$. This is a toy dataset meant to aid in visualizing things.

Let us look at a dataset of size $n = 10$:

$$D = \{(x_1, y_1), \cdots, (x_{10}, y_{10})\}$$



We can turn this into a linear regression problem by forming non-linear polynomial features. Let us assume that we settle for a third degree polynomial. The feature vector is:

$$\mathbf{x} = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

$\mathbf{X}$ is going to be a $d \times n$ matrix with $d = 4$ and $n = 10$. $d = 4$ since we are also adding an intercept. The label vector is $\mathbf{y}$. Concretely,

$$\mathbf{X} = \begin{bmatrix} 1 & \cdots & 1 \\ x_1 & \cdots & x_{10} \\ x_1^2 & \cdots & x_{10}^2 \\ x_1^3 & \cdots & x_{10}^3 \end{bmatrix}, \ \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_{10} \end{bmatrix}$$

The weight vector is going to be:

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

Therefore, our model is:

$$\begin{aligned} f(x) &= w_0 + w_1 x + w_2 x^2 + w_3 x^3 \\ &= \mathbf{w}^T \mathbf{x} \end{aligned}$$

Remark: As far as the original problem is concerned, f is a non-linear function of the feature. After adding more (polynomial) features, f is linear in the weights. This is still called a linear regression problem as far the weights are concerned.

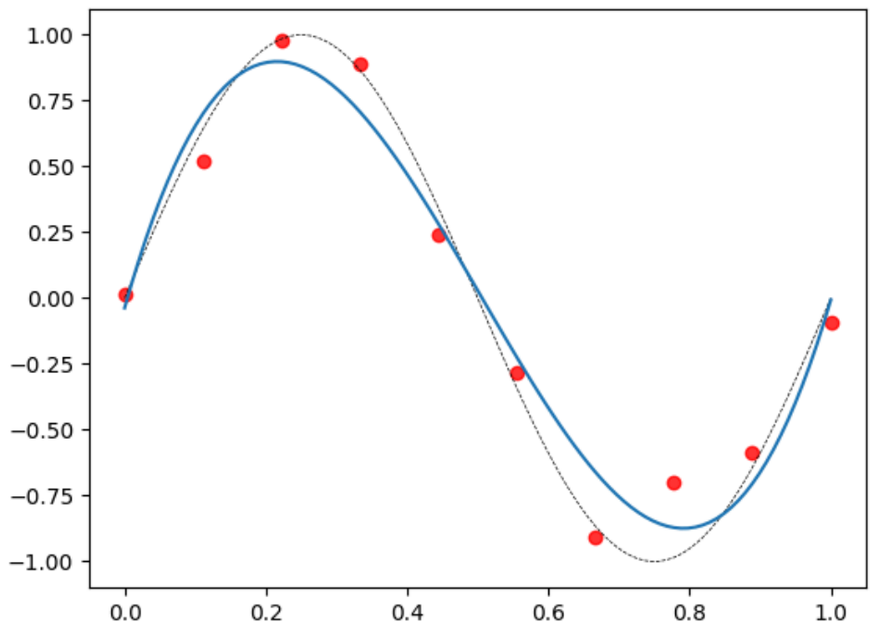
We can now learn the optimal weights by minimizing the SSE. We get:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}\mathbf{X}^T\right)^{-1} \mathbf{X}\mathbf{y}$$

The weights we end up with are:

$$\widehat{\mathbf{w}} = \begin{bmatrix} -0.04 \\ 9.54 \\ -28.12 \\ 18.61 \end{bmatrix}$$

If we plot the function learnt, this is how it looks:

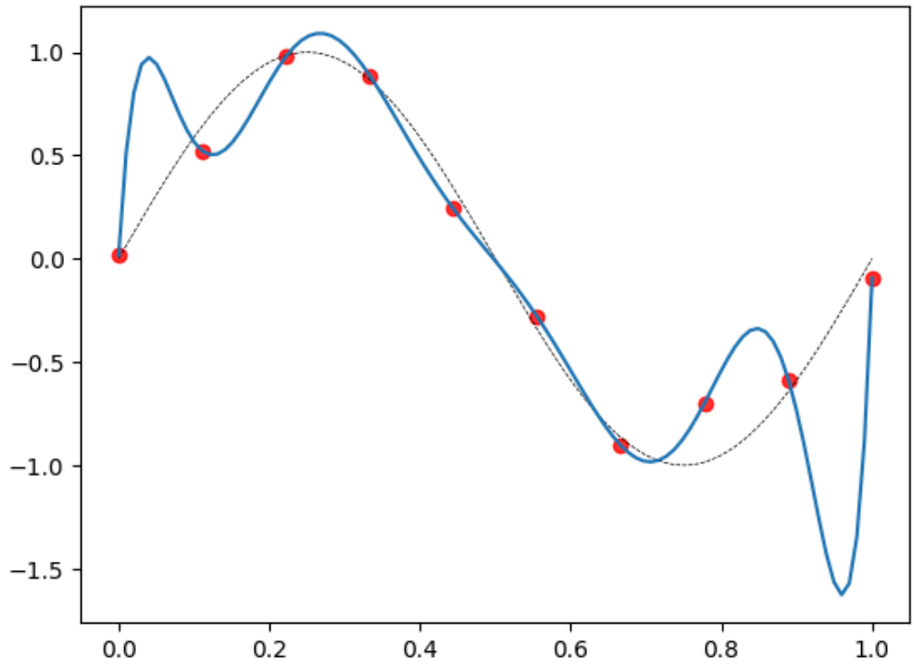


It seems like a reasonable fit.

3.2. Overfitting

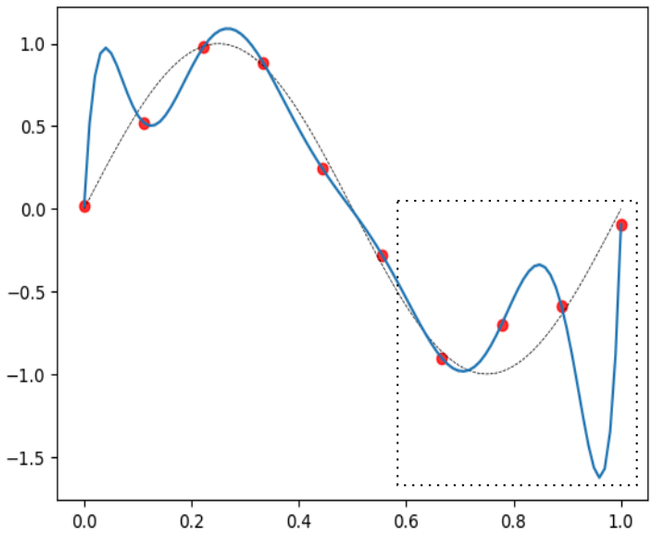
What if we change $p = 9$. The resulting data-matrix will be of shape $d \times n$ with $d = 10, n = 10$.

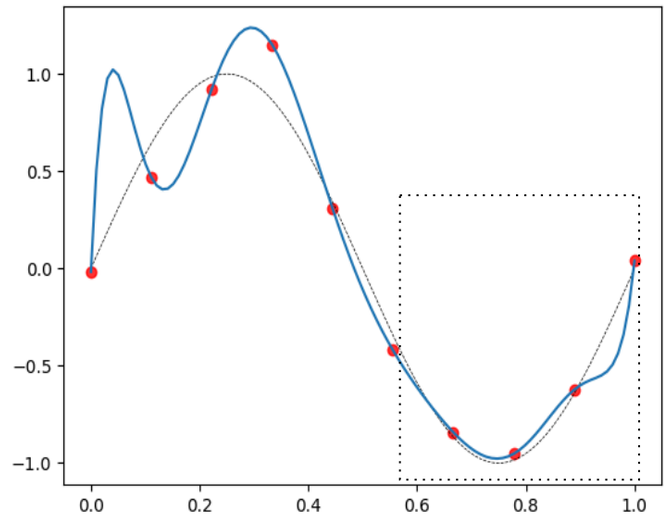
$$\begin{aligned} f(x) &= w_0 + w_1 x + \cdots + w_9 x^9 \\ &= \mathbf{w}^T \mathbf{x} \end{aligned}$$



$$\widehat{\mathbf{w}} = \begin{bmatrix} 0.02 \\ 62.12 \\ -1383.34 \\ 13019.05 \\ -63619.93 \\ 178687.61 \\ -300179.44 \\ 297927.91 \\ -160994.34 \\ 36480.26 \end{bmatrix}$$

The model fits the noise in addition to the signal. This problem is termed **overfitting**. Also note the really high magnitudes for the components of the weight vector. An overfit model is said to be a **high variance model**: small changes in the dataset can result in wildly different models. For example:

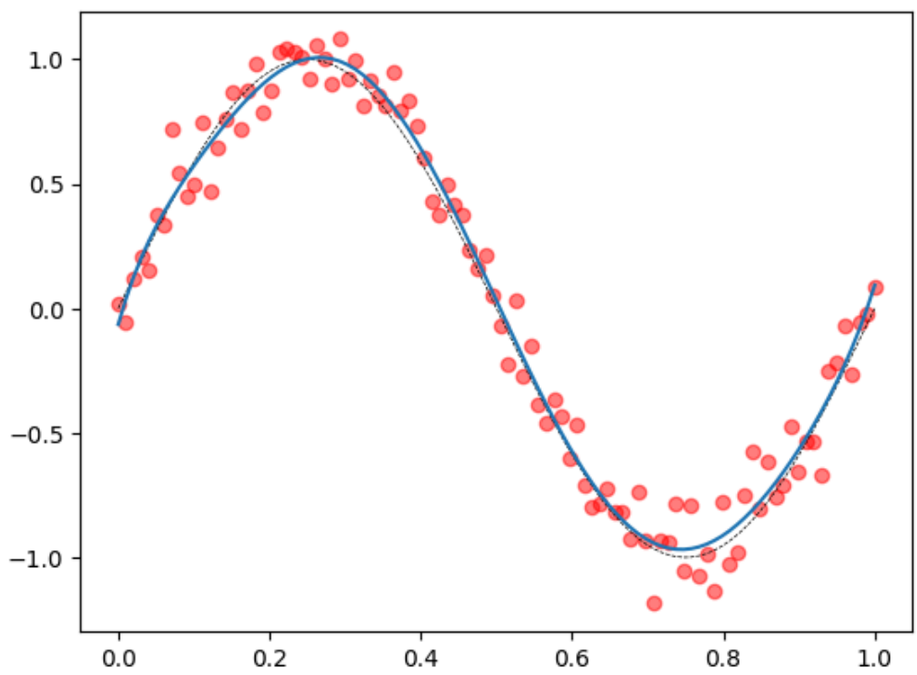




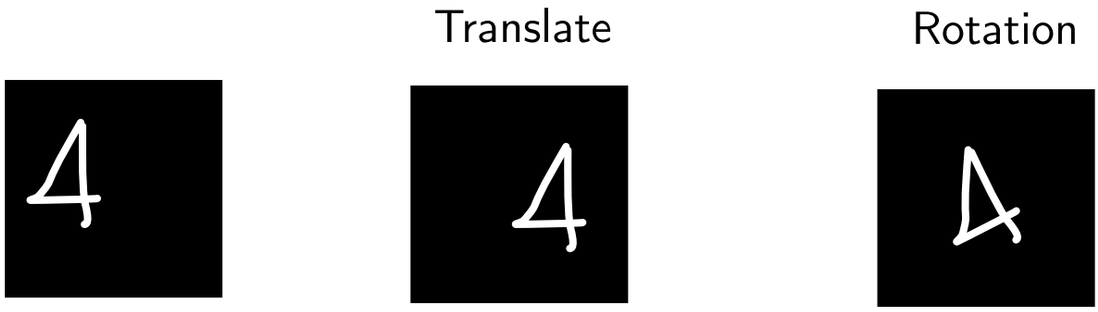
Note the boxed region. A small change in the position of one or two data-points caused a big change in the fitted curve.

3.3. Data Augmentation

Increasing the data or augmenting it (data-augmentation) is one way of mitigating the problem of overfitting. For $n = 100$, the result is as follows:



Data Augmentation is a popular technique for increasing the size of the dataset:



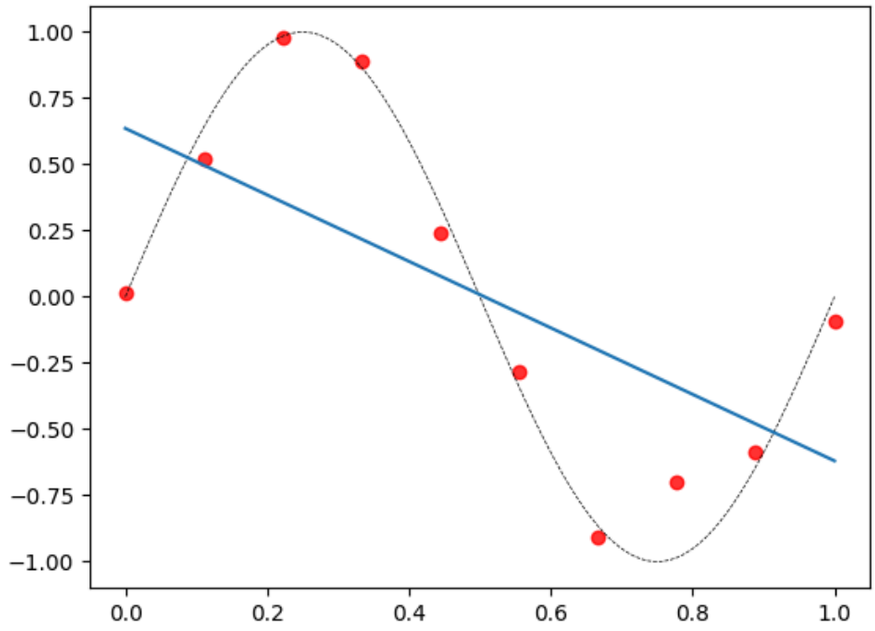
Coming back to the regression problem, the weight vector for this model is:

$$\widehat{\mathbf{w}} = \begin{bmatrix} -0.06 \\ 10.91 \\ -85.89 \\ 609.29 \\ -2557.32 \\ 5953.79 \\ -8253.17 \\ 6949.81 \\ -3318.79 \\ 691.52 \end{bmatrix}$$

Note the relatively smaller values for the weights. Models that are really complex tend to overfit. A high-degree polynomial is one such example.

3.4. Underfitting

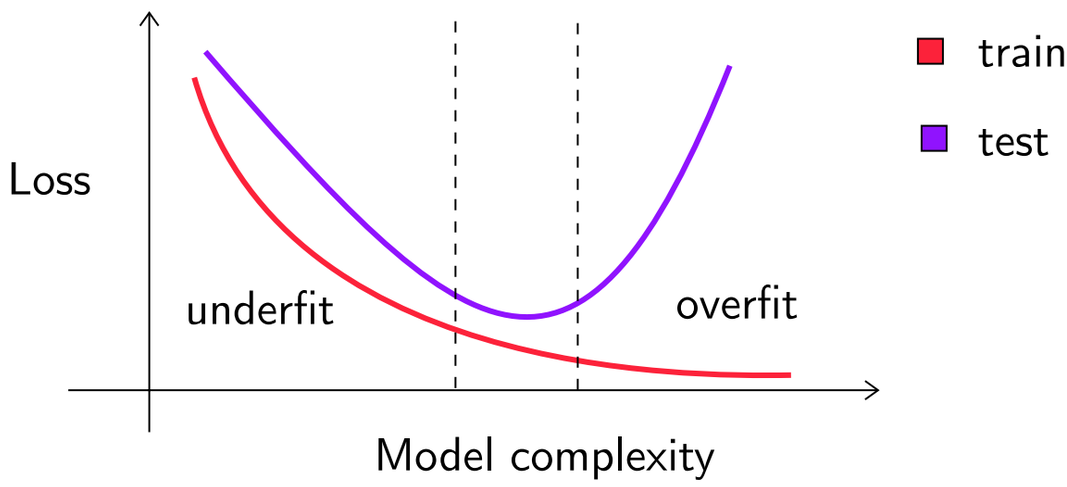
Underfitting is the opposite problem of being unable to capture the signal in the data:



A line is too simple for the given data and completely misses the patterns. When a model underfits, it is usually a good idea to fit a more complex model.

4. Model Complexity

If the train loss is low and the test loss is high, the model is overfitting. If the train loss is itself high, the model is underfitting. Here is a general plot of train and test loss with respect to model complexity:



Note how the training loss keeps decreasing as the models get more and more complex.

5. Regularization

Regularization is a method to mitigate overfitting. We can't always resort

to adding more training data. In such situations, we impose an a constraint on the model's weights. We study two forms of regularization:

5.1. Ridge Regression

The loss function is modified as follows. This is called the regularized loss function:

$$L(\mathbf{w}) = \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2}_{\text{SSE}} + \underbrace{\frac{\lambda}{2} ||\mathbf{w}||_2^2}_{\text{regularization term}} \tag{1}$$

where:

$$||\mathbf{w}||_2^2 = w_1^2 + \cdots + w_d^2$$

There are two aspects to the regularized loss:

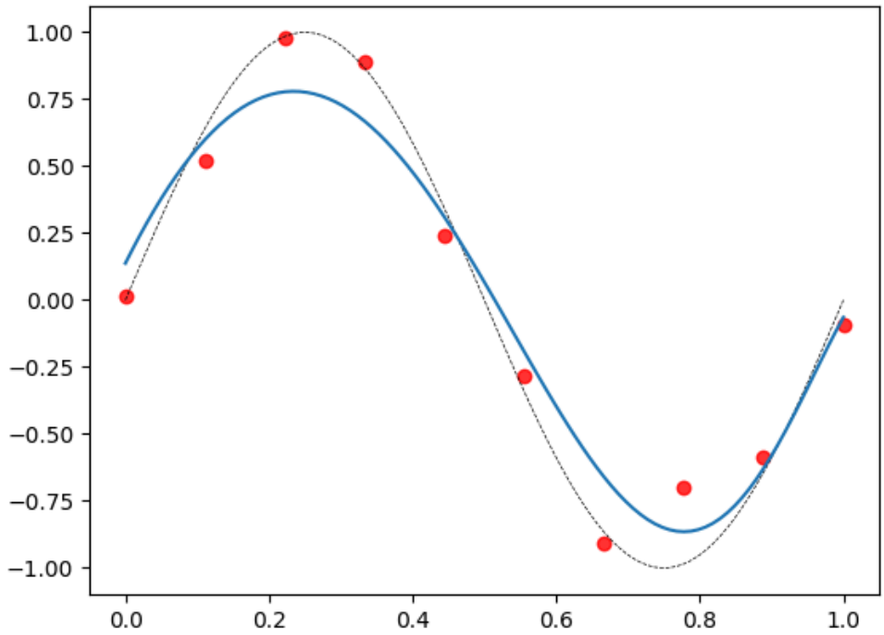
- data-dependent loss: SSE
- model-dependent loss: regularization term
  - controls the complexity of the model

$||\mathbf{w}||_2$ is the L_2 norm of $\mathbf{w}$. This is the usual Euclidean norm. λ is called the regularization rate and is a **hyperparameter**.

<u>Hyperparameters versus Parameters</u>

- parameters are learnt from data
- hyperparameters are chosen prior to the learning
  - however, the best hyperparameter for a given problem is still determined by making use of the (validation) data

For $\lambda = 0.001$, this is the resulting model:



The corresponding weight vector:

$$\widehat{\mathbf{w}} = \begin{bmatrix} 0.14 \\ 5.38 \\ -10.52 \\ -4.02 \\ 2.33 \\ 4.89 \\ 4.44 \\ 2.22 \\ -0.81 \\ -4.10 \end{bmatrix}$$

Ridge regression has the effect of pushing the weights nearer to zero though not exactly to zero. Let us see how to solve for $\mathbf{w}$:

$$L(\mathbf{w}) = \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2}_{\text{SSE}} + \underbrace{\frac{\lambda}{2} ||\mathbf{w}||_2^2}_{\text{regularization term}}$$

On setting the gradient to zero we have

$$\mathbf{X}\mathbf{X}^T\mathbf{w} - \mathbf{X}\mathbf{y} + \lambda\mathbf{w} = 0$$

Simplifying:

$$\left(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I}\right)\mathbf{w} = \mathbf{X}\mathbf{y}$$

Therefore:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I}\right)^{-1}\mathbf{X}\mathbf{y}$$

<u>Geometric view</u>

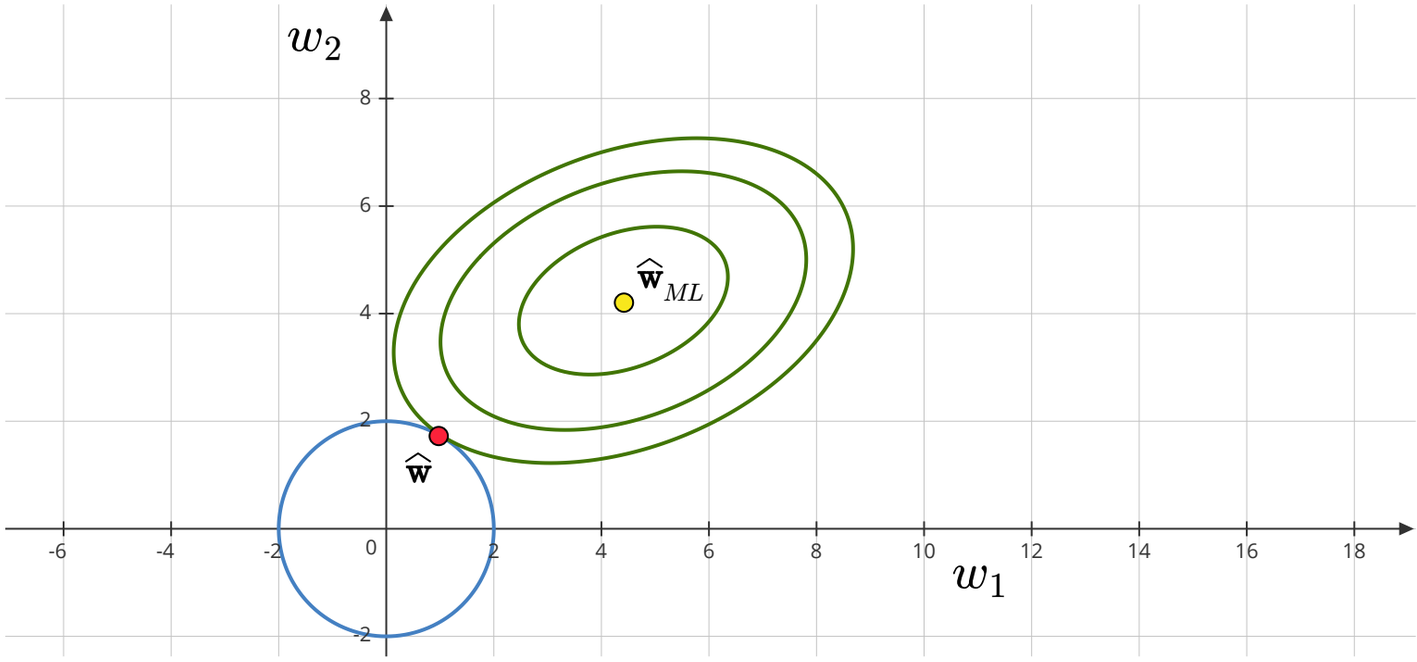
The optimization problem can be equivalently formulated as:

$$\min_{\mathbf{w}} \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T\mathbf{x}_i - y_i\right)^2}_{\text{SSE}}$$

subject to

$$||\mathbf{w}||^2 \leqslant \theta$$

where θ is some scalar dependent on λ. Plotting the contours of the SSE and the constraint region:



<u>Contours of the SSE</u>

$$||\mathbf{X}^T\mathbf{w} - \mathbf{y}||^2 = ||\mathbf{X}^T\widehat{\mathbf{w}}_{ML} - \mathbf{y}||^2 + c$$

From the normal equations, we have $\mathbf{X}\mathbf{X}^T\widehat{\mathbf{w}}_{ML} = \mathbf{X}\mathbf{y}$. Using this, we get:

$$(\mathbf{w} - \widehat{\mathbf{w}}_{ML})^T\left(\mathbf{X}\mathbf{X}^T\right)(\mathbf{w} - \widehat{\mathbf{w}}_{ML}) = c$$

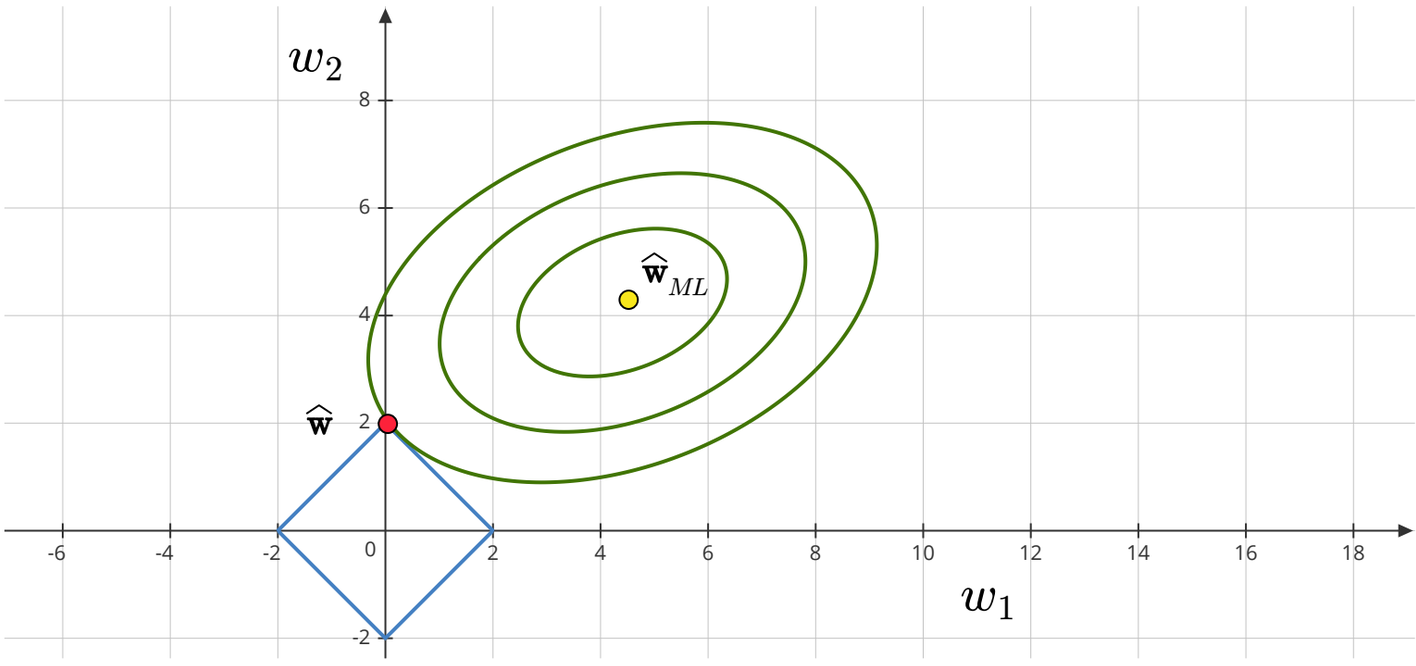
5.2. LASSO Regression

$$L(\mathbf{w}) = \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T\mathbf{x}_i - y_i\right)^2}_{\text{SSE}} + \underbrace{\lambda||\mathbf{w}||_1}_{\text{regularization term}}$$

where

$$||\mathbf{w}||_1 = |w_1| + \cdots + |w|_d$$

is the L_1 norm. It is also the Manhattan distance of the point $\mathbf{w}$ from the origin in $\mathbb{R}^d$. No closed form solution can be found for LASSO regression.



With a high probability, the contours of the SSE will hit one of the corners of the square. This results in a sparse solution.

LASSO: Least Absolute Shrinkage Selection Operator

- Least: we are solving a minimization problem
- Absolute: the L_1 norm uses the sum of the absolute values
- Shrinkage: the weights are going to become smaller
- Selection: since some weights go to zero, it acts as a feature selector

6. Cross Validation

As mentioned earlier, λ is called hyperparameter. It is chosen using a process called cross validation.

6.1. $k-$Fold cross validation

Fold-1	Fold-2	$\cdots$	Fold-k

<u>Algorithm</u>

- For λ in $\left[10^{-4}, 10^{-3}, \cdots, 10^{2}\right]$
  - For k rounds:
    * Train on $k-1$ folds and test on the remaining fold.
      - The size of the training dataset is $\frac{n}{k}(k-1)$
      - The size of the validation dataset is $\frac{n}{k}$
      - Store the validation loss
    * Average the validation losses from the k models
- Return λ that gives least average validation loss

6.2. Leave one out cross validation

<u>Algorithm</u>

- For λ in $\left[10^{-4}, 10^{-3}, \cdots, 10^{2}\right]$
  - For n rounds:
    * Train on $n-1$ data-points and test on the remaining fold.
      · The size of the training dataset is $n-1$
      · The size of the validation dataset is 1
      · Store the validation loss
    * Average the validation losses from the n models
- Return λ that gives least average validation loss

This process of determining the best value of λ for a given problem is called hyperparameter tuning.

7. Probabilistic view

We have:

$$\mathbf{y} \sim N\left(\mathbf{X}^T\mathbf{w}, \sigma^2\mathbf{I}\right)$$

<u>Prior</u>

$$\mathbf{w} \sim N\left(\mathbf{0}, \gamma^2\mathbf{I}\right)$$

$$P(\mathbf{w}) = \frac{1}{\left(\gamma\sqrt{2\pi}\right)^d} \exp\left[\frac{-||\mathbf{w}||^2}{2\gamma^2}\right]$$

<u>Likelihood</u>

$$L(\mathbf{w}; D) = \frac{1}{\left(\sigma\sqrt{2\pi}\right)^d} \exp\left[\frac{-||\mathbf{y} - \mathbf{X}^T\mathbf{w}||^2}{2\sigma^2}\right]$$

<u>Posterior</u>

$$\text{Posterior} \propto \text{Prior} \times \text{Likelihood}$$

$$P(\mathbf{w} \mid D) \propto \exp\left[\frac{-||\mathbf{y} - \mathbf{X}^T\mathbf{w}||^2}{2\sigma^2} + \frac{-||\mathbf{w}||^2}{2\gamma^2}\right]$$

<u>MAP</u>

To get a point estimate for $\mathbf{w}$, we maximize the posterior. This is called the maximum a posteriori estimate. Taking $\log$ for convenience:

$$\max_{\mathbf{w}} \frac{-||\mathbf{y} - \mathbf{X}^T\mathbf{w}||^2}{2\sigma^2} + \frac{-||\mathbf{w}||^2}{2\gamma^2}$$

Maximizing this is the same as minimizing the negative of the objective:

$$\min_{\mathbf{w}} \frac{||\mathbf{y} - \mathbf{X}^T\mathbf{w}||^2}{2\sigma^2} + \frac{||\mathbf{w}||^2}{2\gamma^2}$$

Multiplying throughout by σ^2 and setting $\lambda = \dfrac{\sigma^2}{\gamma^2}$, we have:

$$\min_{\mathbf{w}} \frac{1}{2}||\mathbf{y} - \mathbf{X}^T\mathbf{w}||^2 + \frac{\lambda}{2}||\mathbf{w}||^2$$

This is the same as the ridge regression problem. Thus the MAP estimate with a zero mean Gaussian prior is equivalent to the solution to the ridge

regression. The inverse relationship between λ and γ can be understood as follows:

- Smaller the value of γ, tinier the region in which we would like $\mathbf{w}$ to lie.
- Alternatively, higher the value of λ, the smaller we want the norm of the weights to be.